
INSA - STI 4A - RiSF
Année universitaire 2025 - 2026

RAPPORT DE PROJET

Infrastructure de capteurs : broker MQTT & trans- mission LoRa sécurisée

BARRY Boubacar Sidy
Nhat Lam NGUYEN

Sujet : *Fiche INSA RiSF LoRa - Question 6*
25 juin 2026

Résumé

Ce rapport présente la réalisation d'une infrastructure de capteurs sans fil combinant LoRa et MQTT. Un premier ESP32 joue le rôle de capteur LoRa et publie périodiquement des mesures chiffrées. Un second ESP32 agit comme passerelle : il reçoit les trames LoRa, vérifie leur authenticité, déchiffre les données et les republie vers un broker MQTT. Enfin, un client Python exécuté sur Linux s'abonne au broker, vérifie les messages reçus et permet de publier des trames de test ou de mettre à jour les clés de sécurité. Le protocole applique un chiffrement AES-CBC et une authentification par HMAC-SHA1 afin de protéger la confidentialité, l'intégrité et l'origine des données.

Table des matières

1	Introduction	4
1.1	Contexte	4
1.2	Objectifs	4
1.3	Organisation du rapport	4
2	Architecture générale de la solution	5
2.1	Vue d'ensemble	5
2.2	Matériel et environnement logiciel	5
2.3	Format des trames échangées	5
3	Module capteur (ESP32 #1)	7
3.1	Fichier de configuration <code>config.py</code>	7
3.1.1	Identité et secrets partagés	8
3.1.2	Brochage <code>device_config</code>	8
3.1.3	Paramètres radio <code>lora_parameters</code>	8
3.2	Programme principal <code>main.py</code>	9
3.2.1	Imports et organisation	11
3.2.2	<code>init_lora()</code> - initialisation du module radio	11
3.2.3	<code>build_payload()</code> - génération de la mesure	11
3.2.4	<code>listen_key_update()</code> - fenêtre de réception des clés	12
3.2.5	<code>run()</code> - boucle principale	12
4	Module passerelle (ESP32 #2)	13
4.1	Fichier de configuration <code>config.py</code>	13
4.2	Programme principal <code>main.py</code>	14
4.2.1	<code>init_lora()</code> - une différence clé avec le capteur	18
4.2.2	<code>connect_wifi()</code> - connexion robuste	18
4.2.3	<code>connect_mqtt()</code> - identité et abonnement	18
4.2.4	<code>publish_status()</code> et <code>set_screen()</code> - fonctions utilitaires	19
4.2.5	<code>on_mqtt_message()</code> - traitement des commandes	19
4.2.6	<code>publish_lora_message()</code> - publication MQTT enrichie	19
4.2.7	<code>run()</code> - boucle principale	19
5	Client Python - supervision et sécurité	21
5.1	Constantes : <code>config.py</code>	21
5.2	Primitives cryptographiques : <code>security.py</code>	22
5.3	Empaquetage, messages et rotation des clés	23
5.4	Interface interactive : <code>shell.py</code>	24
5.5	Point d'entrée : <code>main.py</code>	25
5.5.1	<code>on_message()</code> - traitement et rollback automatique	25
5.5.2	Architecture asynchrone	26

6	Sécurisation des échanges	27
6.1	Chiffrement AES des données	27
6.2	Authentification et intégrité des messages	27
6.3	Protocole de mise à jour sécurisée des clés	27
6.4	Analyse de sécurité	27
7	Résultats expérimentaux	28
7.1	Mise en service de la chaîne complète	28
7.2	Réception et déchiffrement des messages sécurisés	28
7.3	Mise à jour des clés	28
7.4	Rejet d'un message altéré	28
7.5	Désactivation de la vérification	28
7.6	Rotation avec mauvaises clés et rollback automatique	29
7.7	Autres outils de test disponibles	29
7.8	Synthèse	29
8	Conclusion	31
A	Annexes	32
A.1	Code source complet	32
A.2	Schémas de câblage	32
A.3	Captures d'écran	32

Table des figures

2.1	Architecture générale de l'infrastructure	5
A.1	Démarrage du client interactif et premier échange capteur → passerelle (termi- naux MicroPython à gauche et à droite, client Python en haut)	32
A.2	Mode <code>listen</code> - réception, vérification HMAC et déchiffrement de messages sécu- risés (<code>[OK] capteur1 {...}</code>)	33
A.3	Commande <code>update_keys</code> - rotation des clés relayée à la passerelle puis confirmée par le capteur (<code>[CONFIRMED]</code>)	33
A.4	Rejet d'un message altéré au sein du flux (<code>[BAD] message rejected</code>), les mes- sages suivants correctement signés continuant d'être acceptés	34
A.5	Commande <code>nocheck on</code> - désactivation volontaire de la vérification HMAC côté passerelle pour observer la réception brute (<code>NO-CHK</code>)	34
A.6	Rotation de clés avec un mauvais HMAC (<code>[BAD_KEYS]</code>), <code>[ROLLBACK]</code> automatique vers les clés précédentes, puis commande <code>status</code> récapitulant l'état courant de la passerelle	35

Liste des tableaux

2.1	Format de message applicatif	6
-----	--	---

Introduction

1.1 Contexte

Le projet s'inscrit dans le cadre du cours Réseaux Sans Fil de l'INSA. L'objectif est de mettre en œuvre une chaîne complète de communication IoT : génération d'une mesure par un capteur, transmission radio longue portée via LoRa, réception par une passerelle, puis diffusion des données vers un broker MQTT. Le broker permet ensuite à un client distant de consulter les mesures et de vérifier que les messages reçus sont bien authentiques et intègres.

1.2 Objectifs

Les objectifs principaux sont les suivants :

- transmettre des données capteur entre deux ESP32 à l'aide de LoRa ;
- republier les données reçues vers un broker MQTT ;
- chiffrer les messages avec AES-CBC ;
- ajouter une authentification par HMAC-SHA1 ;
- vérifier côté passerelle puis côté client Linux que les messages n'ont pas été modifiés ;
- permettre une mise à jour sécurisée des clés AES et HMAC depuis le client Python.

1.3 Organisation du rapport

Le rapport décrit d'abord l'architecture générale, puis le fonctionnement détaillé - fichier par fichier - du capteur, de la passerelle et du client Linux. Les mécanismes de sécurité sont ensuite analysés avant de présenter les tests, les difficultés rencontrées et les améliorations possibles.

Architecture générale de la solution

2.1 Vue d'ensemble

L'architecture comprend quatre éléments principaux :

1. un ESP32 **capteur** qui génère une mesure et l'envoie en LoRa ;
2. un ESP32 **passerelle** qui reçoit la trame LoRa, la vérifie et la republie ;
3. un **broker MQTT** distant (`serveur.iot.com` ou `p-fb.net`) ;
4. un **client Python** sous Linux qui s'abonne au topic MQTT et vérifie les messages.

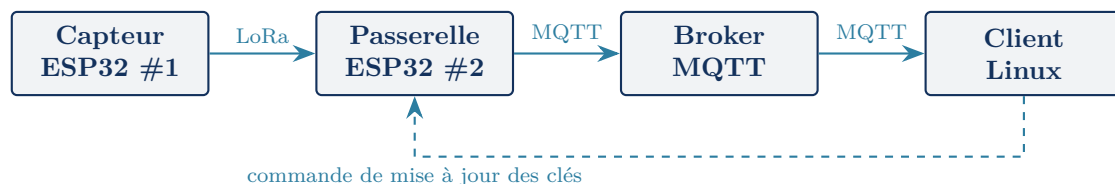


FIGURE 2.1 – Architecture générale de l'infrastructure

2.2 Matériel et environnement logiciel

- 2 cartes ESP32 ;
- 2 modules LoRa SX127x ;
- MicroPython sur ESP32 ;
- Broker MQTT : `serveur.iot.com` ou `p-fb.net`, port 1883 ;
- Client Linux : Python 3 avec `paho-mqtt` et `cryptography`.

2.3 Format des trames échangées

Les messages LoRa sont encodés en base64. Le format logique est :

`node_id | nonce + ciphertext_AES | HMAC.`

Le HMAC est calculé sur :

`INSARISF1 | node_id | nonce + ciphertext_AES.`

La clé AES fait 16 octets et le mode utilisé est AES-CBC. Le nonce correspond à l'IV AES de 16 octets. Le HMAC utilise SHA-1 et produit un tag de 20 octets.

TABLE 2.1 – Format de message applicatif

Champ	Taille	Description
<code>node_id</code>	variable	Identifiant du nœud émetteur
<code>nonce</code>	16 octets	IV AES aléatoire
<code>ciphertext</code>	multiple de 16 octets	Données chiffrées AES-CBC
<code>HMAC-SHA1</code>	20 octets	Intégrité et authenticité
Encodage final	base64	Transport LoRa et MQTT

Module capteur (ESP32 #1)

Ce chapitre détaille le code embarqué qui s'exécute sur le premier ESP32, jouant le rôle de **capteur LoRa**. Son unique mission est de produire une mesure, de la sécuriser, puis de l'émettre en LoRa - il ne se connecte ni au Wi-Fi, ni au broker MQTT.

3.1 Fichier de configuration config.py

Projet/firmware/capteur/config.py

```
1 from machine import Pin, SoftSPI
2
3 WIFI_SSID = 'INSA_RiSF'
4 WIFI_PASSWORD = 'insa2026'
5
6 MQTT_SERVER = 'serveur.iot.com'
7 MQTT_PORT = 1883
8 MQTT_USER = 'esp32'
9 MQTT_PASSWORD = 'esp32'
10 MQTT_DATA_TOPIC = b'LamBouba'
11 MQTT_COMMAND_TOPIC = b'echange/command/passerelle'
12
13 NODE_ID = 'capteur1'
14 AES_KEY = b'0123456789abcdef'
15 HMAC_KEY = b'abcd'
16
17 device_config = {
18     'miso': 19,
19     'mosi': 27,
20     'ss': 18,
21     'sck': 5,
22     'dio_0': 26,
23     'reset': 14,
24     'led': 25,
25 }
26
27 lora_parameters = {
28     'frequency': 868E6,
29     'tx_power_level': 2,
30     'signal_bandwidth': 125E3,
31     'spreading_factor': 11,
32     'coding_rate': 5,
33     'preamble_length': 8,
34     'implicit_header': True,
35     'sync_word': 0x12,
36     'enable_CRC': True,
37     'invert_IQ': False,
```

38 }

💡 Un gabarit commun pour les deux nœuds

Le fichier `config.py` du capteur partage la même structure que celui de la passerelle. Cette conception facilite la maintenance : modifier une constante (fréquence LoRa, clé AES) dans un gabarit unique suffit pour les deux firmwares. Les constantes réseau (`WIFI_SSID`, `MQTT_*`) sont définies ici et activement utilisées par la passerelle.

3.1.1 Identité et secrets partagés

- `NODE_ID` identifie le capteur dans les trames applicatives (`capteur1`) ; il doit être unique sur le réseau LoRa.
- `AES_KEY` (16 octets) est la clé symétrique utilisée pour chiffrer la charge utile en AES-CBC. Elle doit être strictement identique sur le capteur, la passerelle et le client Linux.
- `HMAC_KEY` sert à authentifier les messages *et* les commandes de mise à jour des clés. Elle doit être identique sur le capteur, la passerelle et le client Linux, et peut être mise à jour dynamiquement via le protocole de rotation des clés (voir chapitre 6).

3.1.2 Brochage `device_config`

Ce dictionnaire décrit le câblage SPI entre l'ESP32 et le module radio SX127x :

- `sck`, `mosi`, `miso` : les trois lignes de l'horloge et des données du bus SPI logiciel (`SoftSPI`) ;
- `ss` : la ligne de sélection de puce (*chip select* / NSS) du module LoRa ;
- `dio_0` : la broche d'interruption du SX127x, utilisée par le pilote pour détecter la fin d'une réception ou d'une émission sans avoir à interroger le registre en continu ;
- `reset` : la broche de réinitialisation matérielle du module radio ;
- `led` : une broche libre pour une éventuelle LED de diagnostic.

3.1.3 Paramètres radio `lora_parameters`

Ces paramètres définissent la couche physique LoRa et **doivent être identiques** sur le capteur et sur la passerelle, sous peine de ne jamais pouvoir communiquer :

- `frequency` : 868 MHz, bande ISM européenne autorisée pour le LoRa ;
- `tx_power_level` : puissance d'émission demandée à l'amplificateur du SX127x ;
- `signal_bandwidth` : largeur de bande du signal (125 kHz), qui conditionne le débit symbole ;
- `spreading_factor` : facteur d'étalement (ici SF11). Plus il est élevé, plus la portée et la résistance au bruit augmentent, mais plus le temps d'émission (*airtime*) d'un paquet s'allonge ;
- `coding_rate` : taux de codage correcteur d'erreurs (ici 4/5), qui ajoute de la redondance pour corriger les erreurs de transmission ;
- `preamble_length` : nombre de symboles de préambule envoyés avant les données utiles, nécessaires à la synchronisation du récepteur ;
- `implicit_header` : à `True`, l'en-tête LoRa (longueur, coding rate, présence de CRC) n'est pas transmis dans la trame radio - émetteur et récepteur doivent donc partager *a priori* exactement la même configuration ;
- `sync_word` : mot de synchronisation (0x12) qui permet de filtrer les trames appartenant à un autre réseau LoRa utilisant la même fréquence ;

- `enable_CRC` : active un contrôle d'intégrité au niveau de la couche physique, en plus du HMAC applicatif;
- `invert_IQ` : inversion I/Q, désactivée ici (utile uniquement pour certains modes spécifiques comme le LoRaWAN *downlink*).

3.2 Programme principal `main.py`

Projet/firmware/capteur/main.py

```

1 import os
2 import sys
3 os.chdir("/capteur")
4 from machine import Pin, SoftSPI
5 from time import sleep, ticks_ms, ticks_diff
6 import machine
7 import ujson
8
9 from config import *
10 from security import *
11 from sx127x import SX127x
12 from oled import *
13
14
15 def init_lora():
16     device_spi = SoftSPI(
17         baudrate=10000000,
18         polarity=0,
19         phase=0,
20         bits=8,
21         firstbit=SoftSPI.MSB,
22         sck=Pin(device_config['sck'], Pin.OUT, Pin.PULL_DOWN),
23         mosi=Pin(device_config['mosi'], Pin.OUT, Pin.PULL_UP),
24         miso=Pin(device_config['miso'], Pin.IN, Pin.PULL_UP)
25     )
26     return SX127x(
27         device_spi,
28         pins=device_config,
29         parameters=lora_parameters
30     )
31
32
33 def build_payload(counter):
34     value = 20.0 + counter * 0.5
35     return ujson.dumps({
36         'node': NODE_ID,
37         'type': 'sensor',
38         'counter': counter,
39         'temperature': value,
40         'battery': max(0, 90 - counter)
41     })
42
43
44 def listen_key_update(lora, oled, screen, timeout_ms=2000):
45     global AES_KEY
46     global HMAC_KEY
47 
```

```
48     # Passage explicite en mode reception active
49     lora.receive()
50
51     start_time = ticks_ms()
52     packet_detected = False
53
54     # Attente active non-bloquante du paquet (evite le sleep brut)
55     while ticks_diff(ticks_ms(), start_time) < timeout_ms:
56         if lora.received_packet():
57             print("Msg recu")
58             packet_detected = True
59             break
60             sleep(0.05)
61
62     if not packet_detected:
63         # Forcer le retour au mode veille pour pouvoir emettre au
64         prochain tour
65         lora.sleep()
66         return False
67
68     # Lecture du paquet de mise a jour des cles
69     packet = lora.read_payload()
70
71     # Securite : verification de la signature HMAC du paquet d'update
72     ok, new_aes_key, new_hmac_key = verify_key_update(packet, HMAC_KEY
73     )
74
75     if ok:
76         AES_KEY = new_aes_key
77         HMAC_KEY = new_hmac_key
78
79         screen[0] = 'KEYS OK'
80         screen[1] = NODE_ID
81         screen[2] = 'updated'
82         write_screen(oled, screen)
83         lora.sleep()
84         return True
85
86     # Paquet recu mais invalide (mauvais HMAC, rejeu...)
87     screen[0] = 'KEYS BAD'
88     screen[1] = NODE_ID
89     screen[2] = 'Rejected!'
90     write_screen(oled, screen)
91     lora.sleep()
92     return False
93
94 def run():
95     global AES_KEY
96     global HMAC_KEY
97
98     lora = init_lora()
99     oled, screen = init_oled()
100     counter = 0
101
102     screen[0] = 'LoRa capteur'
103     screen[1] = NODE_ID
```

```

103     screen[2] = 'AES+HMAC'
104     write_screen(oled, screen)
105     sleep(2)
106
107     while True:
108         counter += 1
109         payload = build_payload(counter)
110
111         # Chiffrement AES-CBC + Signature HMAC-SHA1 + Encodage Base64
112         packet = pack_message(NODE_ID, payload.encode(), AES_KEY,
                               HMAC_KEY)
113
114         # Envoi de la trame Base64 via LoRa
115         lora.println(packet)
116         print("[TX] Paquet envoye : {}".format(packet[:30] + "..."))
117
118         screen[0] = 'TX {}'.format(NODE_ID)
119         screen[1] = packet[:16]
120         screen[2] = 'counter {}'.format(counter)
121         write_screen(oled, screen)
122
123         # Fenetre d'ecoute des cles (6 secondes d'ouverture)
124         print("Enter listening")
125         listen_key_update(lora, oled, screen, timeout_ms=6000)
126
127 if __name__ == '__main__':
128     run()

```

3.2.1 Imports et organisation

Le script importe massivement les modules locaux avec `from ... import *(config, security, oled)`. Cette pratique, déconseillée dans un projet Python classique, est courante en MicroPython embarqué : elle évite de préfixer chaque constante et limite la quantité de code source à compiler en mémoire flash. Toutes les constantes de `config.py` (`NODE_ID`, `AES_KEY`, `lora_parameters...`) et toutes les fonctions de `security.py` (`pack_message`, `verify_key_update...`) deviennent ainsi directement accessibles dans `main.py`.

3.2.2 `init_lora()` - initialisation du module radio

Le bus SPI n'est pas le bus matériel de l'ESP32 mais un bus **logiciel** (SoftSPI), ce qui permet de choisir librement les broches `sck/mosi/miso` dans `device_config` plutôt que d'être contraint par le brochage SPI matériel par défaut. Le pilote `SX127x` est ensuite instancié avec ce bus, le dictionnaire de broches et les paramètres radio : il configure les registres du module (fréquence, SF, BW, sync word...) lors de sa construction.

3.2.3 `build_payload()` - génération de la mesure

Cette fonction simule un capteur physique : la température suit une rampe linéaire (`20.0 + counter * 0.5`) et la batterie décroît avec le compteur, sans jamais devenir négative grâce à `max(0, ...)`. Le résultat est sérialisé en JSON avec `ujson.dumps`, l'implémentation JSON allégée de MicroPython. Dans une version finale, cette fonction serait remplacée par une vraie lecture de capteur (DHT22, BME280, etc.) sans changer le reste du programme.

3.2.4 listen_key_update() - fenêtre de réception des clés

Le capteur est *half-duplex* : il ne peut pas émettre et écouter en même temps. Pour permettre malgré tout à la passerelle de lui transmettre de nouvelles clés, le capteur ouvre une courte fenêtre d'écoute après chaque émission :

1. `lora.receive()` arme le module en mode réception ;
2. une boucle d'attente active interroge `lora.received_packet()` toutes les 50 ms, bornée par `ticks_ms()/ticks_diff()` : cette approche est préférée à un simple `sleep()` bloquant car elle permet de sortir de la boucle dès qu'un paquet arrive, sans attendre inutilement tout le `timeout_ms` ;
3. si rien n'est reçu avant l'expiration du délai, le module repasse en veille (`lora.sleep()`) afin de libérer le canal pour la prochaine émission ;
4. si un paquet est reçu, `verify_key_update()` vérifie son HMAC *avant* d'accepter quoi que ce soit ;
5. en cas de **succès** : les variables globales `AES_KEY` et `HMAC_KEY` sont réaffectées avec les nouvelles valeurs, puis le capteur envoie un accusé de réception explicite `KEYS_OK` chiffré et signé avec les *nouvelles* clés via `pack_message()` et `lora.println()` ;
6. en cas d'**échec** (HMAC invalide, format incorrect) : le capteur répond par un NACK `KEYS_FAILED` signé avec ses *anciennes* clés toujours actives, ce qui permet à la passerelle et au client Linux de détecter le désaccord et de déclencher un rollback automatique ;
7. dans tous les cas, l'écran OLED affiche le résultat (`KEYS OK` ou `KEYS BAD`) pour un diagnostic visuel immédiat.

💡 Accusé de réception avec les nouvelles clés

Le capteur applique les nouvelles clés *avant* d'envoyer l'ACK, de sorte que le message `KEYS_OK` est lui-même signé avec les nouvelles clés. Le client Linux, qui tente de le déchiffrer avec les nouvelles clés, peut ainsi confirmer que le capteur les a bien adoptées. Si la rotation a échoué, le capteur renvoie `KEYS_FAILED` signé avec les anciennes clés, ce qui déclenche le rollback automatique côté client.

3.2.5 run() - boucle principale

Après une phase d'initialisation (LoRa, écran OLED, affichage de démarrage), le capteur entre dans une boucle infinie qui, à chaque itération :

1. incrémente le compteur de mesures ;
2. construit la charge utile JSON avec `build_payload()` ;
3. la sécurise avec `pack_message()` (chiffrement AES-CBC puis HMAC-SHA1 puis base64, détaillé au chapitre 5) ;
4. émet le paquet en LoRa avec `lora.println()` ;
5. met à jour l'écran OLED avec le début du paquet encodé ;
6. ouvre une fenêtre d'écoute de 6 secondes via `listen_key_update()` avant de recommencer.

Le cycle complet (émission puis écoute) se répète donc indéfiniment, sans délai fixe supplémentaire : la période d'émission effective est d'environ 6 secondes, dictée par la durée de la fenêtre d'écoute des clés.

Module passerelle (ESP32 #2)

La passerelle est le seul nœud qui parle à la fois LoRa et MQTT : elle reçoit les trames radio du capteur, vérifie leur authenticité, puis les republie sur le broker. Elle joue donc le rôle de **client MQTT** pour le reste de l'infrastructure.

4.1 Fichier de configuration config.py

Projet/firmware/passerelle/config.py

```
1 from machine import Pin, SoftSPI
2
3 WIFI_SSID = 'INSA_RiSF'
4 WIFI_PASSWORD = 'insa2026'
5
6 MQTT_SERVER = 'serveur.iot.com'
7 MQTT_PORT = 1883
8 MQTT_USER = 'esp32'
9 MQTT_PASSWORD = 'esp32'
10 MQTT_DATA_TOPIC = b'LamBouba'
11 MQTT_COMMAND_TOPIC = b'echange/command/passerelle'
12
13 NODE_ID = 'passerelle1'
14 AES_KEY = b'0123456789abcdef'
15 HMAC_KEY = b'abcd'
16
17 device_config = {
18     'miso': 19,
19     'mosi': 27,
20     'ss': 18,
21     'sck': 5,
22     'dio_0': 26,
23     'reset': 14,
24     'led': 25,
25 }
26
27 lora_parameters = {
28     'frequency': 868E6,
29     'tx_power_level': 2,
30     'signal_bandwidth': 125E3,
31     'spreading_factor': 11,
32     'coding_rate': 5,
33     'preamble_length': 8,
34     'implicit_header': True,
35     'sync_word': 0x12,
36     'enable_CRC': True,
37     'invert_IQ': False,
```


38 }

Ce fichier reprend exactement la même structure que celui du capteur (chapitre 3), avec trois différences importantes :

- NODE_ID vaut `passerelle1` ;
- MQTT_DATA_TOPIC (`LamBouba`) est ici réellement utilisé : c'est le topic sur lequel la passerelle publie les données reçues du capteur ;
- MQTT_COMMAND_TOPIC (`echange/command/passerelle`) est également utilisé : la passerelle s'y abonne pour recevoir les commandes de mise à jour des clés envoyées par le client Linux.

💡 Synchronisation des paramètres radio

Les valeurs de `lora_parameters` sont **rigoureusement identiques** entre le capteur et la passerelle : même fréquence, même facteur d'étalement, même bande passante. C'est cette synchronisation qui permet aux deux modules SX127x de communiquer, le LoRa exigeant que l'émetteur et le récepteur partagent exactement la même configuration physique.

4.2 Programme principal `main.py`

📁 `Projet/firmware/passerelle/main.py`

```

1  import os
2  os.chdir("/passerelle")
3  from machine import Pin, SoftSPI
4  from time import sleep
5  import machine
6  import network
7  import ubinascii
8  import ujson
9
10 from config import *
11 from security import *
12 from sx127x import SX127x
13 from oled import *
14 from umqttsimple import MQTTClient
15
16
17 mqtt_client = None
18 no_check_mode = False
19
20
21 def init_lora():
22     device_spi = SoftSPI(
23         baudrate=10000000,
24         polarity=0,
25         phase=0,
26         bits=8,
27         firstbit=SoftSPI.MSB,
28         sck=Pin(device_config['sck'], Pin.OUT, Pin.PULL_DOWN),
29         mosi=Pin(device_config['mosi'], Pin.OUT, Pin.PULL_UP),
30         miso=Pin(device_config['miso'], Pin.IN, Pin.PULL_UP)
31     )
32     modem = SX127x(

```

```
33     device_spi ,
34     pins=device_config ,
35     parameters=lora_parameters
36 )
37 # Force le modem en mode ecoute LoRa des le demarrage
38 modem.receive()
39 return modem
40
41
42 def connect_wifi():
43     wlan = network.WLAN(network.STA_IF)
44
45     # Nettoyage preventif de l'etat interne de la puce Wi-Fi
46     if wlan.active():
47         wlan.active(False)
48         sleep(0.5)
49
50     wlan.active(True)
51     sleep(0.5)
52
53     if wlan.isconnected():
54         print('WiFi connected')
55         print(wlan.ifconfig())
56         return True
57
58     print('Connecting WiFi {}'.format(WIFI_SSID))
59
60     try:
61         wlan.connect(WIFI_SSID, WIFI_PASSWORD)
62     except OSError as e:
63         print("WiFi HW Error: {}. Retrying...".format(e))
64         machine.reset()
65
66     for _ in range(30):
67         if wlan.isconnected():
68             print('WiFi connected')
69             print(wlan.ifconfig())
70             return True
71         sleep(1)
72
73     print('WiFi connection failed')
74     return False
75
76
77 def connect_mqtt():
78     global mqtt_client
79
80     client_id = ubinascii.hexlify(machine.unique_id())
81     mqtt_client = MQTTClient(
82         client_id=client_id,
83         server=MQTT_SERVER,
84         port=MQTT_PORT,
85         user=MQTT_USER,
86         password=MQTT_PASSWORD
87     )
88     mqtt_client.set_callback(on_mqtt_message)
89     mqtt_client.connect()
```

```

90     mqtt_client.subscribe(MQTT_COMMAND_TOPIC)
91     print('Connected to MQTT {}'.format(MQTT_SERVER))
92     return mqtt_client
93
94
95 def publish_status(topic, event, ok, detail=''):
96     payload = ujson.dumps({
97         'node': NODE_ID,
98         'event': event,
99         'ok': ok,
100        'detail': detail
101    })
102    mqtt_client.publish(topic, payload)
103
104
105 def set_screen(lines):
106     if 'screen' not in globals() or 'oled' not in globals():
107         return
108     for index, line in enumerate(lines):
109         screen[index] = line
110     write_screen(oled, screen)
111
112
113 def send_key_update(command_packet):
114     lora.println(command_packet)
115     lora.receive()
116
117
118 def on_mqtt_message(topic, msg):
119     global AES_KEY
120     global HMAC_KEY
121     global no_check_mode
122
123     actual_topic = topic.decode() if type(topic) == bytes else topic
124     config_topic = MQTT_COMMAND_TOPIC.decode() if type(
125         MQTT_COMMAND_TOPIC) == bytes else MQTT_COMMAND_TOPIC
126
127     if actual_topic != config_topic:
128         return
129
130     # Commande NOCHECK (activer/desactiver la verification HMAC)
131     if msg.startswith(NOCHECK_PREFIX):
132         ok, state = verify_nocheck_command(msg, HMAC_KEY)
133         if ok:
134             no_check_mode = state
135             set_screen(['NO CHECK', 'ON' if state else 'OFF'])
136             mqtt_client.publish(MQTT_DATA_TOPIC, b"nocheck_updated")
137             return
138
139     print("[MQTT] Ordre de Key Update recu.")
140     ok, new_aes_key, new_hmac_key = verify_key_update(msg, HMAC_KEY)
141
142     if ok:
143         AES_KEY = new_aes_key
144         HMAC_KEY = new_hmac_key
145         mqtt_client.publish(MQTT_DATA_TOPIC, b"keys_updated")
146         if 'lora' in globals():

```

```
146         send_key_update(msg)
147         lora.receive()
148         set_screen(['KEYS OK', NODE_ID, 'updated'])
149         print("[REKEY] Cles mises a jour avec succes et relayees.")
150     else:
151         mqtt_client.publish(MQTT_DATA_TOPIC, b"keys_rejected")
152         set_screen(['KEYS BAD', NODE_ID])
153         print("[REKEY] Echec : HMAC invalide.")
154 def publish_lora_message(node_id, plaintext, rssi, snr, secure):
155     try:
156         data_str = plaintext.decode() if type(plaintext) == bytes else
157             str(plaintext)
158     except Exception:
159         data_str = str(plaintext)
160
161     payload = ujson.dumps({
162         'node': node_id,
163         'data': data_str,
164         'rssi': rssi,
165         'snr': snr,
166         'secure': secure
167     })
168     mqtt_client.publish(MQTT_DATA_TOPIC, payload)
169
170 def run():
171     global lora
172     global oled
173     global screen
174
175     if not connect_wifi():
176         return
177
178     connect_mqtt()
179     lora = init_lora()
180     oled, screen = init_oled()
181
182     screen[0] = 'Passerelle'
183     screen[1] = NODE_ID
184     screen[2] = 'MQTT + LoRa'
185     write_screen(oled, screen)
186
187     while True:
188         try:
189             mqtt_client.check_msg()
190         except OSError:
191             connect_mqtt()
192
193         if lora.received_packet():
194             packet = lora.read_payload()
195             rssi = lora.packet_rssi()
196             snr = lora.packet_snr()
197             mqtt_client.publish(MQTT_DATA_TOPIC, packet)
198
199             if no_check_mode:
200                 screen[0] = 'RX RAW'
201                 screen[1] = packet[:16].decode() if type(packet) ==
```

```

202         bytes else packet[:16]
203         screen[2] = 'RSSI {}'.format(rssi)
204         screen[3] = 'SNR {}'.format(snr)
205         screen[4] = 'NO-CBK'
206     else:
207         ok, node_id, plaintext = unpack_message(packet,
208         AES_KEY, HMAC_KEY)
209         if ok:
210             screen[0] = 'RX {}'.format(node_id)
211             screen[1] = plaintext.decode()[:16]
212             screen[2] = 'RSSI {}'.format(rssi)
213             screen[3] = 'SNR {}'.format(snr)
214             screen[4] = 'OK'
215         else:
216             screen[0] = 'RX BAD'
217             screen[4] = 'HMAC ERR'
218         write_screen(oled, screen)
219
220         lora.receive()
221
222         sleep(0.1)
223
224 if __name__ == '__main__':
225     run()

```

4.2.1 init_lora() - une différence clé avec le capteur

La configuration matérielle est identique à celle du capteur, mais `init_lora()` appelle ici `modem.receive()` immédiatement après la construction du pilote. La passerelle n'émet jamais de mesure : son rôle est d'écouter le canal LoRa en continu, elle doit donc rester armée en réception dès le démarrage.

4.2.2 connect_wifi() - connexion robuste

Cette fonction va au-delà d'un simple `wlan.connect()` :

- elle désactive puis réactive l'interface Wi-Fi (`wlan.active(False)` puis `True`) afin de repartir d'un état propre, utile après un *soft-reset* de l'ESP32 ;
- elle court-circuite la tentative de connexion si le module est déjà connecté ;
- l'appel à `wlan.connect()` est protégé par un `try/except OSError` : certaines puces Wi-Fi ESP32 lèvent une erreur matérielle transitoire, à laquelle on répond par un `machine.reset()` complet plutôt que de rester bloqué ;
- une boucle de 30 tentatives, espacées d'une seconde, attend la connexion avant d'abandonner et de renvoyer `False`.

4.2.3 connect_mqtt() - identité et abonnement

L'identifiant de connexion MQTT (`client_id`) est dérivé de `machine.unique_id()`, un identifiant matériel propre à chaque puce ESP32, converti en chaîne hexadécimale. Cela garantit un `client_id` unique même si plusieurs passerelles partagent le même `config.py` : un broker MQTT déconnecte en effet le premier client dès qu'un second se reconnecte avec le même identifiant. La bibliothèque `umqttsimple` (un client MQTT minimaliste pour MicroPython) gère ensuite la connexion TCP, l'authentification par utilisateur/mot de passe, puis l'abonnement immédiat au topic de commande `MQTT_COMMAND_TOPIC`.

4.2.4 publish_status() et set_screen() - fonctions utilitaires

`publish_status()` encapsule la publication d'un évènement de diagnostic au format JSON. `set_screen()` met à jour l'écran OLED de façon défensive : elle vérifie d'abord que les variables globales `oled` et `screen` existent avant d'écrire, ce qui évite un plantage si la fonction est appelée (par exemple depuis le callback MQTT) avant la fin de l'initialisation dans `run()`.

4.2.5 on_mqtt_message() - traitement des commandes

Cette fonction est enregistrée comme *callback* et déclenchée par `umqttsimple` chaque fois qu'un message arrive sur un topic souscrit. Elle gère deux types de commandes :

Commandes de mise à jour des clés (KEYUPDATE)

1. elle compare le topic reçu au topic de commande attendu, en normalisant les deux côtés en chaînes de caractères (`topic` est parfois transmis en `bytes` par la bibliothèque MQTT) ; elle ignore silencieusement tout message qui ne concerne pas ce topic ;
2. elle délègue la vérification cryptographique à `verify_key_update()` définie dans `security.py`, qui recalcule le HMAC de la commande avec la clé HMAC **actuelle** et la compare en temps constant ;
3. en cas de succès : les nouvelles clés sont appliquées immédiatement (`global AES_KEY`, `global HMAC_KEY`), un accusé `keys_updated` est publié sur le topic de données, puis la commande est **relayée telle quelle vers le capteur par LoRa** via `send_key_update()` ;
4. en cas d'échec, un accusé `keys_rejected` est publié et les clés en cours **ne sont pas modifiées** : une commande invalide ne peut pas interrompre le service.

Commandes de mode inspection (NOCHECK) La passerelle reconnaît également les commandes préfixées par `NOCHECK_PREFIX`. Leur HMAC est d'abord vérifié par `verify_nocheck_command()` : si la signature est valide, la variable globale `no_check_mode` est basculée à `True` ou `False` selon l'état demandé, et un `nocheck_updated` est publié sur le topic de données. En mode inspection activé, la boucle `run()` affiche les trames reçues sans les vérifier (étiquette `NO-CHK` sur l'OLED), ce qui permet d'observer le trafic brut pendant les tests sans modifier les clés.

💡 Chaîne de confiance bout-en-bout

Le client Linux signe la commande avec la clé HMAC qu'il croit être la clé courante. La passerelle la vérifie avec *sa propre* clé HMAC courante avant de l'appliquer, puis la relaie sans la modifier. Le capteur effectue à son tour la même vérification avec sa propre clé courante. Une commande n'est donc acceptée par un nœud que s'il partageait déjà la bonne clé HMAC au moment de la recevoir - ce qui empêche un tiers ne connaissant pas la clé en cours de forcer une rotation.

4.2.6 publish_lora_message() - publication MQTT enrichie

La fonction `publish_lora_message()` construit un message JSON structuré avant publication sur le broker. Ce JSON regroupe l'identifiant du nœud émetteur, les données utiles déchiffrées, le niveau de signal RSSI, le rapport signal/bruit SNR et un indicateur booléen `secure` précisant si le paquet a bien passé la vérification HMAC. Cette structure facilite le traitement côté abonné MQTT : un client peut filtrer immédiatement les messages sur le champ `secure`, et exploiter RSSI/SNR pour surveiller la qualité du lien radio sans outillage supplémentaire.

4.2.7 run() - boucle principale

Après connexion Wi-Fi (avec retour anticipé si elle échoue), connexion MQTT puis initialisation LoRa et OLED, la passerelle entre dans une boucle infinie qui, à chaque itération :

1. appelle `mqtt_client.check_msg()`, qui interroge le socket MQTT de façon non bloquante et déclenche `on_mqtt_message()` si un message est en attente ; une `OSError` (connexion TCP perdue) entraîne une reconnexion immédiate via `connect_mqtt()` ;
2. vérifie si une trame LoRa est disponible (`lora.received_packet()`) ; si oui, elle lit le paquet, le RSSI et le SNR, publie le paquet brut sur MQTT, puis tente de le déchiffrer pour actualiser l'écran OLED ;
3. réarme systématiquement le module en réception avec `lora.receive()` - une étape **critique** : sans cet appel, le SX127x resterait dans l'état « réception terminée » et ignorerait toute trame suivante ;
4. attend 100 ms (`sleep(0.1)`) avant de recommencer, un compromis entre réactivité et charge CPU.

Client Python - supervision et sécurité

Le client Linux est structuré en **package Python** (Projet/mqtt_client/) composé de quatre fichiers aux responsabilités distinctes :

- `config.py` : toutes les constantes (broker, topics, clés par défaut, préfixes de protocole);
- `security.py` : les primitives cryptographiques (AES-CBC, HMAC-SHA1, pack/unpack, rotation des clés, commande nocheck);
- `shell.py` : la classe `MqttShell` (interface interactive `cmd.Cmd`) et la fonction `print_async()` ;
- `main.py` : le point d'entrée, la connexion MQTT, le traitement des messages et la boucle principale.

Cette séparation permet de réutiliser chaque module indépendamment et d'étendre le client sans toucher au code cryptographique. Le client réimplémente côté CPython les mêmes primitives que `security.py` côté MicroPython, afin que les messages chiffrés sur l'ESP32 puissent être vérifiés sur Linux et inversement.

5.1 Constantes : `config.py`

 `Projet/mqtt_client/config.py`

```
1 BLOCK_SIZE = 16
2 MESSAGE_PREFIX = b'INSARISF1'
3 KEYUPDATE_PREFIX = b'KEYUPDATE'
4 NOCHECK_PREFIX = b'NOCHECK'
5 DEFAULT_BROKER = 'serveur.iot.com'
6 DEFAULT_PORT = 1883
7 DEFAULT_USER = 'esp32'
8 DEFAULT_PASSWORD = 'esp32'
9 DEFAULT_DATA_TOPIC = 'LamBouba'
10 DEFAULT_COMMAND_TOPIC = 'echange/command/passerelle'
11 DEFAULT_AES_KEY = b'0123456789abcdef'
12 DEFAULT_HMAC_KEY = b'abcd'
```

Ce fichier centralise tous les paramètres modifiables en un seul endroit. Le préfixe `NOCHECK_PREFIX` identifie les commandes de mode inspection envoyées à la passerelle. Les constantes `MESSAGE_PREFIX` et `KEYUPDATE_PREFIX` doivent être strictement identiques à celles du firmware, car elles font partie intégrante du calcul HMAC.

5.2 Primitives cryptographiques : security.py

Projet/mqtt_client/security.py - bourrage et HMAC

```

1 def pad(data):
2     padding = BLOCK_SIZE - (len(data) % BLOCK_SIZE)
3     return data + bytes([padding]) * padding
4
5 def unpad(data):
6     padding = data[-1]
7     if padding < 1 or padding > BLOCK_SIZE:
8         raise ValueError('bad padding')
9     return data[:-padding]
10
11 def hmac_sha1(key, message):
12     return hmac.new(key, message, hashlib.sha1).digest()
13
14 def equal_bytes(a, b):
15     return hmac.compare_digest(a, b)

```

- `pad()/unpad()` implémentent un bourrage de type PKCS#7 : on ajoute toujours entre 1 et 16 octets, chacun valant le nombre d'octets ajoutés, ce qui permet de retirer le bourrage sans ambiguïté en lisant le dernier octet ;
- `hmac_sha1()` encapsule `hmac.new(...)` de la bibliothèque standard (RFC 2104) ;
- `equal_bytes()` s'appuie sur `hmac.compare_digest()`, une comparaison en **temps constant** pour résister aux attaques temporelles. Cette précaution est reprise manuellement dans le firmware, MicroPython ne fournissant pas `hmac.compare_digest`.

Projet/mqtt_client/security.py - chiffrement AES-CBC

```

1 def encrypt_aes_cbc(key, plaintext):
2     nonce = os.urandom(BLOCK_SIZE)
3     cipher = Cipher(algorithms.AES(key), modes.CBC(nonce), backend=
4         default_backend())
5     encryptor = cipher.encryptor()
6     ciphertext = encryptor.update(pad(plaintext)) + encryptor.finalize
7         ()
8     return nonce + ciphertext
9
10 def decrypt_aes_cbc(key, data):
11     nonce = data[:BLOCK_SIZE]
12     ciphertext = data[BLOCK_SIZE:]
13     cipher = Cipher(algorithms.AES(key), modes.CBC(nonce), backend=
14         default_backend())
15     decryptor = cipher.decryptor()
16     return unpad(decryptor.update(ciphertext) + decryptor.finalize())

```

Un IV (nonce) de 16 octets aléatoire est généré par `os.urandom()` à chaque chiffrement et préfixé au texte chiffré. Deux messages identiques successifs produisent donc des textes chiffrés différents, empêchant un observateur passif de repérer les répétitions dans le trafic.

5.3 Empaquetage, messages et rotation des clés

Projet/mqtt_client/security.py - pack/unpack

```

1 def pack_message(node_id, plaintext, aes_key, hmac_key):
2     encrypted = encrypt_aes_cbc(aes_key, plaintext)
3     mac_data = MESSAGE_PREFIX + b'|' + node_id.encode() + b'|' +
4         encrypted
5     mac = hmac_sha1(hmac_key, mac_data)
6     packet = node_id.encode() + b'|' + encrypted + b'|' + mac
7     return base64.b64encode(packet).decode()
8
9 def unpack_message(packet, aes_key, hmac_key):
10     try:
11         raw = base64.b64decode(packet)
12         node_id_bytes, rest = raw.split(b'|', 1)
13         node_id = node_id_bytes.decode()
14         encrypted, mac = rest.rsplit(b'|', 1)
15         mac_data = MESSAGE_PREFIX + b'|' + node_id_bytes + b'|' +
16             encrypted
17         expected = hmac_sha1(hmac_key, mac_data)
18         if not equal_bytes(mac, expected):
19             return False, node_id, b''
20         plaintext = decrypt_aes_cbc(aes_key, encrypted)
21         return True, node_id, plaintext
22     except Exception:
23         return False, '', b''

```

`unpack_message()` vérifie le HMAC **avant** de déchiffrer (approche *verify-then-decrypt*). Le `rsplit(b'|', 1)` pour extraire le HMAC (dernier champ) et le `split(b'|', 1)` pour extraire l'identifiant (premier champ) tolèrent un `node_id` contenant accidentellement le séparateur.

✓ Vérification HMAC avant déchiffrement

Vérifier le HMAC avant de déchiffrer empêche un oracle de padding : si l'on déchiffrait d'abord, un message d'erreur de bourrage pourrait renseigner un attaquant sur le contenu du texte chiffré. Seuls les paquets signés avec la bonne clé HMAC sont déchiffrés, sur toute la chaîne.

Projet/mqtt_client/security.py - rotation et nocheck

```

1 def build_key_update(new_aes_key, new_hmac_key, current_hmac_key):
2     nonce = os.urandom(8)
3     command = KEYUPDATE_PREFIX + b'|' + nonce + b'|' + new_aes_key.hex
4         ().encode() + b'|' + new_hmac_key.hex().encode()
5     mac = hmac_sha1(current_hmac_key, command)
6     return command + b'|' + mac.hex().encode()
7
8 def build_nocheck_command(state_bool, current_hmac_key):
9     nonce = os.urandom(8)
10    state = b'1' if state_bool else b'0'
11    command = NOCHECK_PREFIX + b'|' + nonce + b'|' + state
12    mac = hmac_sha1(current_hmac_key, command)
13    return command + b'|' + mac.hex().encode()

```

Le format de la commande de rotation est : KEYUPDATE | nonce(8 octets) | nouvelle_clé_AES(hex)

| nouvelle_clé_HMAC(hex) | HMAC(commande). Le HMAC est calculé avec la clé HMAC **courante**, ce qui garantit que seul le détenteur des secrets actuels peut autoriser une rotation.

`build_nocheck_command()` construit la commande de bascule du mode inspection, également signée avec la clé HMAC courante. Un nonce de 8 octets garantit que deux commandes successives vers le même état produisent des paquets différents.

❓ Rôle du nonce dans les commandes

Le nonce aléatoire de 8 octets empêche un observateur de reconnaître une commande à sa simple structure binaire, et rend impossible la réutilisation d'une commande interceptée.

5.4 Interface interactive : `shell.py`

Projet/mqtt_client/shell.py - MqttShell (extrait)

```

1 class MqttShell(cmd.Cmd):
2     intro = "Client MQTT/LoRa securise interactif."
3     prompt = "(mqtt) "
4
5     def do_listen(self, arg):
6         self.client.subscribe(self.args.data_topic)
7         self.listening = True
8
9     def do_update_keys(self, arg):
10        # Publie la commande de rotation et active le mode double-
11           decodage
12        new_aes_key, new_hmac_key = [a.encode() for a in arg.split()]
13        command = build_key_update(new_aes_key, new_hmac_key, self.
14           args.hmac_key)
15        self.client.publish(self.args.command_topic, command)
16        self.args.pending_update = True
17        self.args.fallback_keys = (self.args.aes_key, self.args.
18           hmac_key)
19        self.args.aes_key = new_aes_key
20        self.args.hmac_key = new_hmac_key
21
22    def do_nocheck(self, arg):
23        state = (arg.strip().lower() == 'on')
24        command = build_nocheck_command(state, self.args.hmac_key)
25        self.client.publish(self.args.command_topic, command)
26
27    def do_status(self, arg):
28        print(f"Broker: {self.args.broker}:{self.args.port}")
29        print(f"Cle AES actuelle: {self.args.aes_key.decode()}")
30        print(f"Cle HMAC actuelle: {self.args.hmac_key.decode()}")

```

La classe `MqttShell` hérite de `cmd.Cmd`, le module de la bibliothèque standard Python pour les interfaces en ligne de commande interactives. Chaque méthode préfixée `do_` définit automatiquement une commande. La boucle `cmdloop()` gère la lecture, la complétion par tabulation et l'aide automatique depuis les docstrings. Les commandes disponibles sont :

- `listen / stop_listen` : s'abonner ou se désabonner du topic de données;
- `update_keys <aes> <hmac>` : lancer une rotation de clés et activer le mode double-décodage;
- `nocheck on|off` : activer ou désactiver la vérification HMAC sur la passerelle;

- `publish <node_id> <message>` : publier un message de test sécurisé ;
- `status` : afficher broker, topic et clés actives ;
- `quit / exit` : fermer le client.

La fonction `print_async()` assure un affichage non-intrusif des messages MQTT reçus en arrière-plan : elle efface la ligne courante, affiche le message, puis réaffiche le prompt et le contenu en cours de saisie via `readline.get_line_buffer()`, de sorte que les notifications entrantes n'interrompent pas la frappe d'une commande.

5.5 Point d'entrée : `main.py`

 `Projet/mqtt_client/main.py - on_message avec double-decodage`

```

1 def on_message(client, userdata, message, args, shell=None):
2     raw_payload = message.payload.decode()
3     SYSTEM_MESSAGES = ("keys_updated", "keys_rejected", "
4         nocheck_updated")
5     if raw_payload in SYSTEM_MESSAGES:
6         print_async(f'\n[SYSTEM] Passerelle: {raw_payload}', shell)
7         return
8
9     pending = getattr(args, 'pending_update', False)
10    valid, node_id, plaintext = unpack_message(message.payload, args.
11        aes_key, args.hmac_key)
12
13    if pending and not valid:
14        # Essai avec les anciennes clés (Trial Decryption)
15        fb_aes, fb_hmac = args.fallback_keys
16        fb_valid, fb_node_id, fb_plaintext = unpack_message(message.
17            payload, fb_aes, fb_hmac)
18        if fb_valid:
19            args.message_count += 1
20            data = fb_plaintext.decode()
21            if data == "KEYS_FAILED" or args.message_count >= 2:
22                # Rollback : restaurer les anciennes clés localement
23                et republier
24                args.aes_key, args.hmac_key = fb_aes, fb_hmac
25                args.pending_update = False
26                command = build_key_update(fb_aes, fb_hmac, args.
27                    active_keys[1])
28                client.publish(args.command_topic, command)
29            elif pending and valid:
30                if plaintext.decode() == "KEYS_OK":
31                    args.pending_update = False # Rotation confirmée
32            elif valid:
33                print_async(f'\n[OK] {node_id} {plaintext.decode()}', shell)
34            else:
35                print_async(f'\n[BAD] message rejected\nraw: {raw_payload}',
36                    shell)

```

5.5.1 `on_message()` - traitement et rollback automatique

Cette fonction est le cœur du traitement des messages MQTT côté client :

1. les messages système de la passerelle (`keys_updated`, `keys_rejected`, `nocheck_updated`) sont affichés directement sans tentative de déchiffrement ;

2. hors rotation en attente, `unpack_message()` est appelé avec les clés courantes ;
3. en mode `pending_update` (rotation demandée mais pas encore confirmée), un **double-décodage** est appliqué : on essaie d'abord les nouvelles clés, puis les anciennes (`fallback_keys`) si les nouvelles échouent ;
4. si deux messages consécutifs sont déchiffrables uniquement avec les anciennes clés, ou si le capteur répond `KEYS_FAILED`, le client déclenche un **rollback automatique** : il restaure ses clés localement et publie une nouvelle commande de rotation vers les anciennes valeurs, à destination de la passerelle ;
5. si le capteur répond `KEYS_OK` déchiffrable avec les nouvelles clés, la rotation est confirmée et `pending_update` est désactivé.

Projet/mqtt_client/main.py - boucle principale

```

1 def main():
2     args = parser.parse_args()
3     client = make_mqtt_client(args)
4     shell = MqttShell(args, client)
5     client.on_message = lambda c, u, m: on_message(c, u, m, args,
6         shell)
7     client.loop_start()      # Thread MQTT en arriere-plan
8     shell.cmdloop()         # Boucle interactive au premier plan
9     client.loop_stop()
10    client.disconnect()

```

5.5.2 Architecture asynchrone

La boucle MQTT est lancée en **arrière-plan** via `client.loop_start()`, qui crée un thread dédié. Le fil principal est ainsi libre pour la boucle interactive `shell.cmdloop()`. Les deux boucles s'exécutent en parallèle : le shell attend la saisie pendant que le thread MQTT reçoit et traite les messages, en appelant `print_async()` pour afficher les notifications sans perturber le prompt.

Format des clés sur la ligne de commande

`parse_bytes()` convertit un argument de ligne de commande directement en octets ASCII (`value.encode()`) : taper `-aes-key 0123456789abcdef` produit les 16 octets de la clé attendue, cohérents avec `DEFAULT_AES_KEY = b'0123456789abcdef'` dans `config.py`. Pour les nouvelles clés transmises par `update_keys`, `build_key_update()` les encode en hexadécimal (`new_aes_key.hex()`) pour les transporter sans ambiguïté de séparateur.

Sécurisation des échanges

6.1 Chiffrement AES des données

Le chiffrement est réalisé en AES-CBC avec une clé de 16 octets. Chaque paquet contient un nonce de 16 octets utilisé comme IV AES. Ce nonce est aléatoire, ce qui évite que deux messages identiques produisent le même texte chiffré.

6.2 Authentification et intégrité des messages

L'intégrité et l'origine du message sont garanties par un HMAC-SHA1. Le HMAC est calculé sur le préfixe `INSARISF1`, l'identifiant du nœud et la partie chiffrée. Cette construction empêche qu'un attaquant modifie le message sans connaître la clé HMAC.

6.3 Protocole de mise à jour sécurisée des clés

Le client Linux peut publier une commande de mise à jour des clés sur le topic de commande. Cette commande contient une nouvelle clé AES, une nouvelle clé HMAC et un HMAC calculé avec la clé HMAC actuelle. La passerelle vérifie la commande, met à jour ses clés puis la transmet au capteur par LoRa. Le capteur vérifie à son tour la commande avant d'appliquer les nouvelles clés.

6.4 Analyse de sécurité

✓ Ce que le protocole protège

Le chiffrement AES-CBC garantit la confidentialité face à une écoute passive du canal radio ou MQTT, et le HMAC-SHA1 garantit qu'un message modifié ou forgé sans connaître la clé sera détecté et rejeté.

✓ Vérification HMAC avant déchiffrement

Le protocole vérifie systématiquement le HMAC **avant** de déchiffrer : cette approche *verify-then-decrypt* évite qu'un attaquant exploite les erreurs de déchiffrement pour en déduire des informations sur le contenu (oracle de padding). Seuls les paquets signés avec la bonne clé HMAC sont déchiffrés, sur l'ensemble de la chaîne : radio LoRa, broker MQTT, et client Linux.

Résultats expérimentaux

7.1 Mise en service de la chaîne complète

Le client Linux interactif (`mqtt_client/main.py`) a été lancé en parallèle des deux firmwares ESP32, chacun affichant son propre journal sur sa liaison série dans un panneau de terminal dédié. La figure A.1 montre cet environnement de test : à gauche le journal du capteur (émission [TX] d'un paquet puis ouverture de la fenêtre d'écoute des clés), à droite celui de la passerelle (réception [RX], niveau RSSI et SNR), et en haut le client Python, qui liste au démarrage l'ensemble des commandes disponibles : `listen`, `update_keys`, `nocheck`, `publish`, `status`, `stop_listen`.

7.2 Réception et déchiffrement des messages sécurisés

La commande `listen` place le client en écoute sur le topic `LamBouba`. Chaque trame brute reçue (MQTT `raw`) est vérifiée puis déchiffrée : en cas de succès, le client affiche `[OK] capteur1` suivi du contenu JSON en clair (température, batterie, compteur). La figure A.2 illustre cette réception pour deux messages consécutifs, avec une température croissante d'une mesure à l'autre, cohérente avec la rampe simulée par `build_payload()`.

7.3 Mise à jour des clés

La commande `update_keys` publie une commande de rotation sur le topic `echange/command/passerelle`. La passerelle applique les nouvelles clés puis relaie la commande au capteur par LoRa, qui confirme à son tour la mise à jour. Le client affiche alors `[SYSTEM] Passerelle: keys_updated` puis `[CONFIRMED] Mise à jour des clés validée par le capteur`, comme le montre la figure A.3. La commande `status` permet ensuite de relire les clés AES et HMAC actuellement actives sur la passerelle.

7.4 Rejet d'un message altéré

Lorsqu'un message ne passe pas la vérification HMAC, le client l'identifie avec `[BAD] message rejected` et affiche la trame brute reçue, sans que cela n'interrompe la réception des messages valides suivants : la figure A.4 montre un message rejeté entouré de deux messages acceptés normalement, illustrant que la vérification s'effectue indépendamment pour chaque trame.

7.5 Désactivation de la vérification

La commande `nocheck on` permet de désactiver volontairement la vérification HMAC côté passerelle, afin d'observer la réception des trames en clair (étiquetées `NO-CHK` sur l'écran de la passerelle et publiées en MQTT sous forme `RX RAW`). La figure A.5 illustre ce mode de diagnostic, utile pour inspecter le contenu brut d'un paquet pendant les tests.

7.6 Rotation avec mauvaises clés et rollback automatique

La figure A.6 illustre le comportement du système lorsqu'une rotation échoue côté capteur : la passerelle relaie la nouvelle clé proposée, mais le capteur la rejette ([BAD_KEYS] (Old Keys Valid)) et continue de signer ses messages avec son ancienne clé HMAC. Le client journalise un [ROLLBACK] et la passerelle revient à son tour aux clés précédentes, ce qui rétablit immédiatement la cohérence entre les trois composants sans interrompre le flux de mesures.

7.7 Autres outils de test disponibles

En complément du mode interactif, le projet fournit des outils en ligne de commande pour des tests rapides ou scriptés. Un test direct du broker, indépendant du chiffrement, peut être réalisé avec `mosquitto_pub` :

```
>_ test de publication MQTT
```

```
mosquitto_pub -h serveur.iot.com -u esp32 -P esp32 -t LamBouba -m  
coucou
```

La sous-commande `publish-test` de `mqtt_client.py` permet de publier un unique paquet sécurisé sans passer par le capteur LoRa :

```
>_ publish-test
```

```
python3 Projet/linux/mqtt_client.py publish-test \  
--broker serveur.iot.com \  
--mqtt-user esp32 \  
--mqtt-password esp32 \  
--data-topic LamBouba \  
--aes-key 0123456789abcdef \  
--hmac-key abcd \  
--node-id capteur1 \  
--message coucou
```

Le script `publish_secure.sh` automatise quant à lui l'envoi d'un flot de plusieurs paquets sécurisés consécutifs :

```
>_ publish_secure.sh
```

```
./publish_secure.sh \  
--broker serveur.iot.com \  
--mqtt-user esp32 \  
--mqtt-password esp32 \  
--topic LamBouba \  
--count 10 \  
--interval 1
```

7.8 Synthèse

L'ensemble de ces tests couvre le cycle de vie complet du protocole : émission et réception normales, rotation de clés réussie, rejet ciblé d'un message altéré au sein d'un flux par ailleurs valide, mode de diagnostic sans vérification, et reprise automatique après une rotation de clés

défaillante. Le système se comporte de façon cohérente sur l'ensemble de ces scénarios.

Conclusion

Le projet permet de mettre en œuvre une chaîne complète IoT LoRa/MQTT avec vérification cryptographique des messages. Le capteur produit des mesures, la passerelle les transporte vers MQTT et le client Linux vérifie les messages reçus. Les mécanismes AES-CBC et HMAC-SHA1 assurent respectivement la confidentialité et l'intégrité des échanges.

Les améliorations principales seraient l'ajout d'un compteur anti-rejeu, le remplacement d'AES-CBC par AES-GCM et l'ajout d'une journalisation plus détaillée côté passerelle. Le script Bash de publication permet désormais de générer et d'envoyer automatiquement un flot de paquets sécurisés avec des messages différents, ce qui facilite les tests de réception et de robustesse.

Annexes

A.1 Code source complet

Les fichiers principaux du projet sont :

- `Projet/firmware/capteur/main.py`, `config.py`, `security.py`;
- `Projet/firmware/passerelle/main.py`, `config.py`, `security.py`;
- `Projet/linux/mqtt_client.py`;
- `publish_secure.sh`.

A.2 Schémas de câblage

Les ESP32 sont câblés aux modules LoRa SX127x en SPI. Les broches utilisées sont définies dans les fichiers `config.py` du capteur et de la passerelle. Les deux modules doivent partager la masse, l'alimentation, SCK, MOSI, MISO, NSS, RST et DIO0.

A.3 Captures d'écran

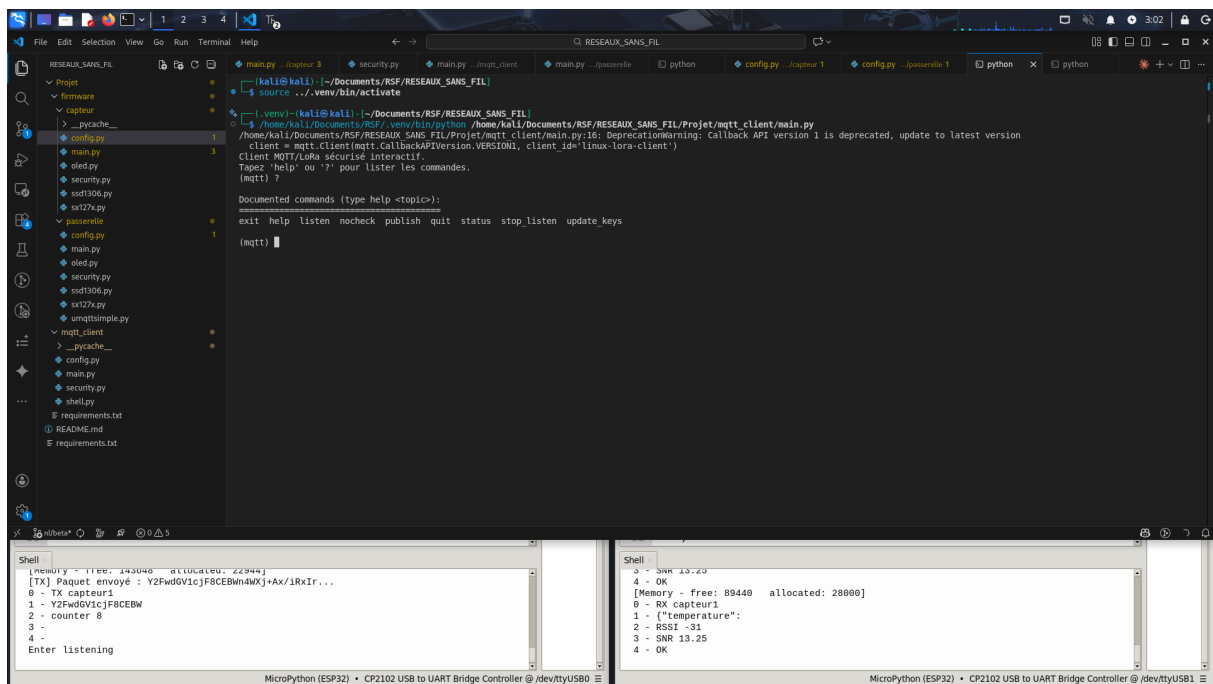


FIGURE A.1 – Démarrage du client interactif et premier échange capteur → passerelle (terminaux MicroPython à gauche et à droite, client Python en haut)

```

kali@kali:~/Documents/RSF/RESEAUX_SANS_FIL$ source ../venv/bin/activate
(.venv)kali@kali:~/Documents/RSF/RESEAUX_SANS_FIL$
(.venv)kali@kali:~/Documents/RSF/RESEAUX_SANS_FIL$ python /home/kali/Documents/RSF/RESEAUX_SANS_FIL/Projet/mqtt_client/main.py
/home/kali/Documents/RSF/RESEAUX_SANS_FIL/Projet/mqtt_client/main.py:16: DeprecationWarning: Callback API version 1 is deprecated, update to latest version
client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION1, client_id='linux-lora-client')
Client MQTT/LoRa sécurisé interactif.
Tapez 'help' ou '?' pour lister les commandes.
(mqtt) ?

Documented commands (type help <topic>):
  exit help listen nocheck publish quit status stop listen update keys

(mqtt) listen
Connecté à serveur.iot.com:1883, en écoute sur le topic LamBouba

[MQTT raw] Y2FwdGVicjF8NTk1fz4yY3B2ZEN4LkVv+ftt8Aes2U1E53gv5UWqr4bY0B40VpepW1cD90fcskF06dy87TnuQA+Kw6YrV8Br/IcM09iFAzN/c3Xco4UNATS2991Ag0RQmh2eqpvo5Dk2An/prvHIIHwYAFj6+RifxH20KwQmWh3Mo9eg=
[OK] capteur1 {"temperature": 28.5, "battery": 73, "node": "capteur1", "counter": 17, "type": "sensor"}

[MQTT raw] Y2FwdGVicjF8ZD90a2taes096re4LUDTL1GaYs9W0X7u6VW9o0pWlrc0evqEu3F31Qhpb0d50PyA44/sj94+IadZ2KZ68trDm0AHdsoc28kmpPjxqBw3v76p08mCpu16Ml5aLLD0XSSc+CfaxoxYrJfKDCXz9CjkgumG66QArLwoY4Zn0Zedg=
[OK] capteur1 {"temperature": 29.0, "battery": 72, "node": "capteur1", "counter": 18, "type": "sensor"}
(mqtt)

```

FIGURE A.2 – Mode listen - réception, vérification HMAC et déchiffrement de messages sécurisés ([OK] capteur1 {...})

```

kali@kali:~/Documents/RSF/RESEAUX_SANS_FIL$ source ../venv/bin/activate
(.venv)kali@kali:~/Documents/RSF/RESEAUX_SANS_FIL$
(.venv)kali@kali:~/Documents/RSF/RESEAUX_SANS_FIL$ python /home/kali/Documents/RSF/RESEAUX_SANS_FIL/Projet/mqtt_client/main.py
/home/kali/Documents/RSF/RESEAUX_SANS_FIL/Projet/mqtt_client/main.py:16: DeprecationWarning: Callback API version 1 is deprecated, update to latest version
client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION1, client_id='linux-lora-client')
Client MQTT/LoRa sécurisé interactif.
Tapez 'help' ou '?' pour lister les commandes.
(mqtt) ?

Documented commands (type help <topic>):
  exit help listen nocheck publish quit status stop listen update keys

(mqtt) listen
Connecté à serveur.iot.com:1883, en écoute sur le topic LamBouba

[MQTT raw] Y2FwdGVicjF8NTk1fz4yY3B2ZEN4LkVv+ftt8Aes2U1E53gv5UWqr4bY0B40VpepW1cD90fcskF06dy87TnuQA+Kw6YrV8Br/IcM09iFAzN/c3Xco4UNATS2991Ag0RQmh2eqpvo5Dk2An/prvHIIHwYAFj6+RifxH20KwQmWh3Mo9eg=
[OK] capteur1 {"temperature": 28.5, "battery": 73, "node": "capteur1", "counter": 17, "type": "sensor"}

[MQTT raw] Y2FwdGVicjF8ZD90a2taes096re4LUDTL1GaYs9W0X7u6VW9o0pWlrc0evqEu3F31Qhpb0d50PyA44/sj94+IadZ2KZ68trDm0AHdsoc28kmpPjxqBw3v76p08mCpu16Ml5aLLD0XSSc+CfaxoxYrJfKDCXz9CjkgumG66QArLwoY4Zn0Zedg=
[OK] capteur1 {"temperature": 29.0, "battery": 72, "node": "capteur1", "counter": 18, "type": "sensor"}
(mqtt) update keys 123456789ABCDEF 2232
Comande de mise à jour des clés publiée sur exchange/command/passerelle
En attente de confirmation du capteur (mode double-décodage activé).

[SYSTEM] Passerelle: keys updated
(mqtt) status
Broker: serveur.iot.com:1883
En écoute: Oui (Topic: LamBouba)
Clé AES actuelle: 123456789ABCDEF
Clé HMAC actuelle: 2232

[MQTT raw] Y2FwdGVicjF8ZFAhW7Wcn5ul1995v+IDRaZ14j52IH8K67Dy5lodVLN87u/c751SHu2n1F0TukatkQNHduQ=
[OK] capteur1 KEYS OK
[CONFIRMED] Mise à jour des clés validée par le capteur.
(mqtt)

```

FIGURE A.3 – Commande update_keys - rotation des clés relayée à la passerelle puis confirmée par le capteur ([CONFIRMED])

```

main.py ...config.py ...python
~/venv/(kali@kali) ~/Documents/RSF/RESEAU_SANS_FIL
$ ./mqtt_client/main.py
Client = mqtt.Client(mqtt.CallbackVersion.VERSION1, client_id='linux-lora-client')
Client MQTT/LoRa sécurisé interfacci.
Tapez 'help' ou '?' pour lister les commandes.
(mqtt) ?
Documented commands (type help <topic>):
exit help listen nocheck publish quit status stop_listen update_keys
(mqtt) listen
Connecté à serveur.iot.com:1883, en écoute sur le topic LamBouba
[MQTT raw] Y2FwdGVicjF8NTk1KkF2Z4f13bZ2E4KkVv+f8t8AesW2U1E3j3v5Uqf4bY0bMA0vpepW1tC090fCskF06d9j77NuDA+Kw6YrV8Br/IocM09iFAZU/C3Xco4UNAT52991AgQ0mhz2eqpvo5Dk0ZAn/pzvHIIHwYAFJ6+RiFtXZ0KwQw3K0w9eg==
[OK] capteur1 ("temperature": 20.5, "battery": 73, "node": "capteur1", "counter": 17, "type": "sensor")
[MQTT raw] Y2FwdGVicjF8ZD90aGtaes986e4LdTL1GaYs9W0x7U6W900pUlc0eocvEu3F31Qhpb0d50PyA44/sj94+IadZKZ68trDm0Ahesoc28kUmPj)xp0w63v76p08mCqwl6sMl5aLLD0X0ScC+cfaxoxYr3FKDCXz9XcJkgumW660Arl9v42n02ED=
[OK] capteur1 ("temperature": 22.0, "battery": 72, "node": "capteur1", "counter": 18, "type": "sensor")
(mqtt) update_keys 123456789ABCDEF 2322
Comande de mise à jour des clés publiée sur échange/command/passerelle
En attente de confirmation du capteur (mode double-décodage activé).
[SYSTEM] Passerelle: keys updated
(mqtt) status
Broker: serveur.iot.com:1883
En écoute: oui (Topic: LamBouba)
Clé AES actuelle: 123456789ABCDEF
Clé HMAC actuelle: 2322
[MQTT raw] Y2FwdGVicjF8DFAHw7Wcn5u11955v+IDRazl4j521HBK67DYs1odVLN87u/C751SHU2n1F0TukatK0HndQ=
[OK] capteur1 KEYS OK
[CONFIRMED] Mise à jour des clés validée par le capteur.
[MQTT raw] DATA[capteur_oxef]5[ba108881799a5db7f6960d8f843e415e]Bad28647e16f150a614cb1d78b4b1dfb2c695261054c8d9b189bacb158041d81e2ba4f3549cc4e802cad7604c7ca61b8d39ffcfba427e14168eb519c9137fdj0c50f4afcd3da5c3e48d8b0be72971d52487f6db
[BAD] message rejected
raw: DATA[capteur_oxef]5[ba108881799a5db7f6960d8f843e415e]Bad28647e16f150a614cb1d78b4b1dfb2c695261054c8d9b189bacb158041d81e2ba4f3549cc4e802cad7604c7ca61b8d39ffcfba427e14168eb519c9137fdj0c50f4afcd3da5c3e48d8b0be72971d52487f6db
[MQTT raw] Y2FwdGVicjF8bnYjdhc1ht391mUGG9qBbR20MqXefY0ubx12iaf1j4h0NE/V3K1Q3J4VxrhCreASRA4cGNhAYtoTVCH6H+oTFwLGuFFZD+0b0f5ZLZ0mUUCnb51CSZL+ILjxgND0dWDL0W6Kum8H1X0BUZ/x3yF37nLE62CBqG4M5/hxy+6CjP89==
[OK] capteur1 ("temperature": 32.0, "battery": 66, "node": "capteur1", "counter": 24, "type": "sensor")
(mqtt)
Shell
[venv] ~ - tree: 143000 allocated: 22912
[TX] Paquet envoyé : Y2FwdGVicjF8bnYjdhc1ht391mUGG9q...
0 - TX capteur1
1 - Y2FwdGVicjF8bnYj
2 - counter 24
3 -
4 -
Enter listening
MicroPython (ESP32) • CP2102 USB to UART Bridge Controller @ jdevttyUSB0

```

FIGURE A.4 – Rejet d’un message altéré au sein du flux ([BAD] message rejected), les messages suivants correctement signés continuant d’être acceptés

```

main.py ...config.py ...python
~/venv/(kali@kali) ~/Documents/RSF/RESEAU_SANS_FIL
$ ./mqtt_client/main.py
Client = mqtt.Client(mqtt.CallbackVersion.VERSION1, client_id='linux-lora-client')
Client MQTT/LoRa sécurisé interfacci.
Tapez 'help' ou '?' pour lister les commandes.
(mqtt) ?
Documented commands (type help <topic>):
exit help listen nocheck publish quit status stop_listen update_keys
(mqtt) listen
Connecté à serveur.iot.com:1883, en écoute sur le topic LamBouba
[MQTT raw] Y2FwdGVicjF8DFAHw7Wcn5u11955v+IDRazl4j521HBK67DYs1odVLN87u/C751SHU2n1F0TukatK0HndQ=
[OK] capteur1 KEYS OK
[CONFIRMED] Mise à jour des clés validée par le capteur.
[MQTT raw] DATA[capteur_oxef]5[ba108881799a5db7f6960d8f843e415e]Bad28647e16f150a614cb1d78b4b1dfb2c695261054c8d9b189bacb158041d81e2ba4f3549cc4e802cad7604c7ca61b8d39ffcfba427e14168eb519c9137fdj0c50f4afcd3da5c3e48d8b0be72971d52487f6db
[BAD] message rejected
raw: DATA[capteur_oxef]5[ba108881799a5db7f6960d8f843e415e]Bad28647e16f150a614cb1d78b4b1dfb2c695261054c8d9b189bacb158041d81e2ba4f3549cc4e802cad7604c7ca61b8d39ffcfba427e14168eb519c9137fdj0c50f4afcd3da5c3e48d8b0be72971d52487f6db
[MQTT raw] Y2FwdGVicjF8bnYjdhc1ht391mUGG9qBbR20MqXefY0ubx12iaf1j4h0NE/V3K1Q3J4VxrhCreASRA4cGNhAYtoTVCH6H+oTFwLGuFFZD+0b0f5ZLZ0mUUCnb51CSZL+ILjxgND0dWDL0W6Kum8H1X0BUZ/x3yF37nLE62CBqG4M5/hxy+6CjP89==
[OK] capteur1 ("temperature": 32.0, "battery": 66, "node": "capteur1", "counter": 24, "type": "sensor")
(mqtt) nocheck on
Comande no-check-on publiée sur échange/command/passerelle
[SYSTEM] Passerelle: nocheck updated
[MQTT raw] Y2FwdGVicjF8bW00slkskf2fVb/B2z5D5y0qHwY0V8Ny0Ts1E58B6G0Yrbhp12090W7Kc1374b0jbc8pbiY04EY/1tq+nH6NhlFqJH2JcH2YU3p0b13LYPpXJe+Y1ynBUUSX1V0P0cV50sU0i4dH07Wd9M6zXTdMhG5jXNZnuHds1ZKq==
[OK] capteur1 ("temperature": 34.0, "battery": 62, "node": "capteur1", "counter": 28, "type": "sensor")
[MQTT raw] Y2FwdGVicjF8GGCRyDv0uM2GQMD75/iaichlPBXZacc05fz1Cs8YbBK0n1I46oyINTK3M4+7H2F6aSEtH/pMB6ELq0b03J3u0A5SGcCZLxP7T0h/v8EYNSP22IDVcs875K83rqs0b0WSP2aYw+TW66GPj6mwQJ0W9VEEufbg94YU7+y8nz4A=
[OK] capteur1 ("temperature": 34.5, "battery": 61, "node": "capteur1", "counter": 29, "type": "sensor")
(mqtt)
Shell
[venv] ~ - tree: 143000 allocated: 22912
[TX] Paquet envoyé : Y2FwdGVicjF8GGCRyDv0uM2GQMD75...
0 - TX capteur1
1 - Y2FwdGVicjF8GGCR
2 - counter 29
3 -
4 -
Enter listening
MicroPython (ESP32) • CP2102 USB to UART Bridge Controller @ jdevttyUSB0

```

FIGURE A.5 – Commande `nocheck on` - désactivation volontaire de la vérification HMAC côté passerelle pour observer la réception brute (NO-CHEK)

The screenshot shows a terminal window with a MQTT client running. The logs show the client receiving messages from a LoRa gateway. The messages contain temperature, battery, node, and counter data. The client also shows the current keys and the status of the gateway. The shell window shows the memory usage of the client, indicating that the client is running on a microcontroller with limited memory.

```

[MQTT raw] Y2PwGv1CjF8n1t0Y1D1S1R7sgkYpABRCBzIn5Wk2tH++91cW19kXJ]SN9U2AU/IA/X8cX7dU0WwfrT+V8Yeo1B0tqvgU4rZ5/nCh9X/LJJf1N0ZK8VBCfsApTxu2gWwEJesLBEVHy9GkCNlCPRggyr+znyzeexAcDlJBgGy0abdcj]Kkx3Pg==
[OK] capteur1 {"temperature": 35.0, "battery": 60, "node": "capteur1", "counter": 30, "type": "sensor"}

[MQTT raw] Y2PwGv1CjF8n1b7K7K7m3S5j5vnxz2S7b5SKQ3C7H69x1+0oxbTEd72dP2NEj084Zlg3A/n/LXo59GQgAcFsb3pV0dU0f825tBDQ7LZbdWt0uxa1GyH5Q5VRQHYm+Q4/k+YYgk+EgBTSD/TFjPmghg+Pd+1D+0YmNQ2+Rp2sT3dsXTQ==
[OK] capteur1 {"temperature": 35.5, "battery": 59, "node": "capteur1", "counter": 31, "type": "sensor"}

[MQTT ?]

Documented commands (type help <topic>):
=====
exit help listen nocheck publish quit status stop_listen update_keys

[MQTT] update keys 123456789ABCDEF 2231
Comande de mise à jour des clés publiée sur échange/command/passerelle
En attente de confirmation du capteur (mode double-décodage activé).

[SYSTEM] Passerelle: keys updated

[MQTT raw] Y2PwGv1CjF8n1t0Y1D1S1R7sgkYpABRCBzIn5Wk2tH++91cW19kXJ]SN9U2AU/IA/X8cX7dU0WwfrT+V8Yeo1B0tqvgU4rZ5/nCh9X/LJJf1N0ZK8VBCfsApTxu2gWwEJesLBEVHy9GkCNlCPRggyr+znyzeexAcDlJBgGy0abdcj]Kkx3Pg==
[BAD_KEYS] (Old Keys Valid) capteur1 {"temperature": 36.5, "battery": 57, "node": "capteur1", "counter": 33, "type": "sensor"} (Message 1/2)

[MQTT raw] Y2PwGv1CjF8n1MhVpnt0ZnK0r2yAHGwmb9Hf1cFqUwKCDIUzRZCQAHayCkJs0f0SAH0YwYmRk1X06tVVCd+ZL/C3C18+y1vRA+loH9qtoovB33ka/xeq9JCsWYx4dfyEY2cGdLh21kR32z3DRASyWf3mD+uQFFxKsEuL+luo17V336vPr1Q==
[ROLLBACK] Annulation de la mise à jour des clés !

[SYSTEM] Passerelle: keys updated

[MQTT raw] Y2PwGv1CjF8w5/akvZM3nUuvrJ1DZABqLA3nY4npCUXC7pdF6a1CV81mWnCJVITG824ZGCFNLZusCNJo=
[OK] capteur1 KEYS_FAILED

[MQTT raw] Y2PwGv1CjF8Bw282Yys/Vtyb/U0D6E1ISgDwQhRE24DfGb6G1Jf101709Rm8LvdKxapx8/dK2pAvGxYV0DqJwrvTggEY3D57HA4dRyOkqfIE/on9z2/NeXv6s2LHSDcNAe40eMw1f/c9RnH0XCG/crzY6QYxc06JrIn5B16a6T4P14d1DboRLw=
[OK] capteur1 {"temperature": 39.5, "battery": 51, "node": "capteur1", "counter": 39, "type": "sensor"}

[MQTT raw] Y2PwGv1CjF8X53d617KX4wJFsn6ZmEmmpsBwvCOP148IjHrVeuH3HLu1IEc+oxnRgcveZj]D4Q1z+paPSlRK1PCOc+F4EDnOUwM5WFnZar/mIXe/Ry+7VeD0G9jEJ]6fsoJjERXOnwqEYvYmGskTfKQ1Aa3ZDeUCw/Fro8tnF8WsbkNmpc0A=
[OK] capteur1 {"temperature": 40.0, "battery": 50, "node": "capteur1", "counter": 40, "type": "sensor"}

[MQTT] status
Broker: serveur.iot.com:1883
En écoute: Oui (Topic: LamBouba)
Clé AES actuelle: 123456789ABCDEF
Clé HMAC actuelle: 2232

Shell
[newy - free: 14300 - allocated: 23024]
[TX] Paquet envoyé : Y2PwGv1CjF8NE294qEHiesyZrz79N...
0 - TX capteur1
1 - Y2PwGv1CjF8NE29
2 - counter 45
3 -
4 -
Enter listening

Shell
3 - SNR 12.73
4 - NO-CHK
[Memory - free: 89392 allocated: 28048]
0 - RX RAW
1 - Y2PwGv1CjF8NE29
2 - RSSI -32
3 - SNR 13.0
4 - NO-CHK

```

FIGURE A.6 – Rotation de clés avec un mauvais HMAC ([BAD_KEYS]), [ROLLBACK] automatique vers les clés précédentes, puis commande **status** récapitulant l'état courant de la passerelle